

LABORATORIO 02

¿Y si se cae uno?

Quórum KRaft y resiliencia.

Guía para hacerlo por tu cuenta, paso a paso

Confluent Platform 8.2.0 · Kafka 4.2 en modo KRaft · 3 brokers

Duración estimada · 60 minutos

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

En el laboratorio 01 encendiste **un** servidor. Hoy vas a construir **tres que se coordinan entre ellos**, y después vas a **matar al que manda** para ver qué hace el clúster sin él.

El plano lo escribes tú. Hay una plantilla con el primer servidor resuelto y dos huecos, y tu trabajo es completarla.

LO QUE ESTE LABORATORIO DEMUESTRA

Un clúster de tres aguanta perder uno. Y el que vuelve no recupera su puesto.

Son dos partes y las dos se ven en pantalla. **Aguanta perder uno**, porque vas a apagar al líder y el clúster va a seguir atendiendo. **Y el que vuelve no recupera su puesto**, porque cuando lo enciendas de nuevo va a entrar como seguidor, y el liderazgo se va a quedar donde está.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Haber hecho el laboratorio 01	Hoy se da por sabido qué es un <code>CLUSTER_ID</code> y qué hace <code>docker compose up -d</code>
Docker Desktop corriendo, con 6 GB de RAM	Hoy son tres servidores. Ajustes de Docker Desktop, pestaña <i>Resources</i>
Git Bash abierto	Es la terminal que usamos en todo el curso
Un editor de texto	VSCode con <code>code</code> , o <code>notepad</code> . Hoy vas a escribir bastante YAML
Los puertos 9092, 9093 y 9094 libres	Uno por servidor

ANTES DE TOCAR NADA · DOS COSAS

Uno. Actualiza el repositorio:

```
cd /c/KAFKA/curso-kafka-sunat
git pull
```

Dos. Baja el laboratorio 01 si lo dejaste arriba, porque ocupa el puerto 9092:

```
docker ps --filter name=kafka-broker
```

El caso

El servidor único que encendiste en el laboratorio 01 funciona. **Y no sirve para producción**, por una razón que no tiene vuelta. Si se cae, se cayó todo.

El director de tecnología de **NovaTech Logistics** lo dice de otra manera. *No me interesa que funcione. Me interesa qué pasa el día que uno de los servidores se apague, porque ese día va a llegar.*

Hoy vas a montar tres y a provocar ese día tú mismo.

LA METÁFORA DEL CURSO · EL RESTAURANTE

De aquí en adelante Kafka es **un restaurante**. El **mozo** que toma y entrega los pedidos es el **broker**, o sea el servidor. Un **tipo de comanda** es el **tópico**. Los **sectores del salón** son las **particiones**. Las **libretas de respaldo** que copian cada comanda son las **réplicas**.

Hoy hay **tres mozos** en el salón, y entre ellos hay uno que es el **jefe de turno**. No atiende más mesas que los demás. Lo que hace es **decidir quién atiende qué sector**, y llevar la cuenta.

Y el jefe de turno no se nombra a dedo. **Los tres mozos lo eligen entre ellos**, y para eso tienen que estar de acuerdo la mayoría. Eso es el **quórum**.

Generar la identidad del clúster

Qué vamos a hacer

Igual que en el laboratorio 01, el clúster necesita un identificador único. La diferencia es que hoy **ese mismo identificador va a ir en los tres servidores**, y tiene que ser idéntico en los tres.

PARA ENTENDERLO MEJOR

Es **el nombre del restaurante bordado en el uniforme**. Los tres mozos llevan el mismo. Si uno trae otro nombre bordado, no trabaja aquí.

El comando

```
cd Capitulo_2/lab-02-validacion-quorum-resiliencia
chmod +x bin/*.sh kafka-cli/*.sh
kafka-cli/generate-cluster-id.sh
```

cd Capitulo_2/lab-02-validacion-quorum-resiliencia — la carpeta del laboratorio. **Todos los comandos de esta guía salen de aquí**

chmod +x — permiso de ejecución para los scripts, que en Git Bash a veces se pierde al clonar

kafka-cli/generate-cluster-id.sh — un script del curso. Por dentro ejecuta `kafka-storage random-uuid`. **No toca ningún clúster ni ningún disco**

LO QUE VAS A VER · DENTRO DE UNA CAJA EXPLICATIVA

YlCN8AhrTfWKst5eUgWhrg

COPIA ESE VALOR AHORA

Lo vas a pegar tres veces en el paso siguiente. Guárdalo en un bloc de notas.

Y usa el tuyo, no el del ejemplo. Cada ejecución genera uno distinto, de 22 caracteres.

VAS BIEN SI...

Tienes una cadena de **22 caracteres** copiada en algún sitio de donde la puedas pegar.

Armar el plano de los tres servidores

Qué vamos a hacer

Este es el paso largo del laboratorio, y el único donde escribes en vez de ejecutar. La plantilla trae **el servidor 1 resuelto** con dos huecos, y los servidores 2 y 3 los agregas tú copiando el patrón.

Copia la plantilla a tu carpeta de trabajo

```
cp plantillas/docker-compose-3-brokers.template.yml mi-cluster/docker-compose.yml
code mi-cluster/docker-compose.yml
```

cp <origen> <destino> — copia la plantilla a `mi-cluster/`, que es tu carpeta de trabajo. **El comando no imprime nada, y en Unix el silencio significa que salió bien**

code <archivo> — lo abre en VSCode. Si no tienes VSCode, usa `notepad mi-cluster/docker-compose.yml`

Cambio 1 · el quórum

Busca esta línea, cerca de la línea 30:

```
KAFKA_CONTROLLER_QUORUM_VOTERS: '{{TODO_VOTERS_3_NODES}}'
```

Reemplaza el marcador por esto, **tal cual, sin espacios de más**:

```
1@kafka-broker-1:39092,2@kafka-broker-2:39093,3@kafka-broker-3:39094
```

CONCEPTO · QUÓRUM DE CONTROLADORES

La lista de **quiénes tienen derecho a votar** cuando hay que elegir jefe de turno. Se escribe como `id@servidor:puerto`, separados por comas. **Tiene que ser idéntica en los tres servidores**, porque si uno tiene una lista distinta, cree que la elección es otra.

POR QUÉ EL PUERTO 39092 Y NO EL 9092

Los servidores hablan entre ellos por una puerta distinta de la que usan los clientes. El **39092** es la puerta de la coordinación, la que usan para votar y ponerse de acuerdo. El **9092** es la puerta de la calle. **Mezclarlas es el error del laboratorio 04**, y ahí lo vas a ver fallar en vivo.

Cambio 2 · la identidad

Busca la línea del `CLUSTER_ID` y pega el tuyo, el del paso 1. Las comillas se quedan:

```
CLUSTER_ID: 'YlCN8AhrTfWKst5eUgWhrg'
```

Cambio 3 · los servidores 2 y 3

Debajo del bloque de `kafka-broker-1` hay un comentario que dice **TU TURNO**. Ahí van los otros dos.

Copia el bloque entero de `kafka-broker-1`, pégalo dos veces, y cambia siete cosas en cada copia. Esta es la tabla, y conviene tenerla al lado mientras escribes.

Qué cambiar	En el 2	En el 3
El nombre del servicio, el <code>hostname</code> y el <code>container_name</code>	<code>kafka-broker-2</code>	<code>kafka-broker-3</code>
<code>ports</code>	<code>"9093:9093"</code>	<code>"9094:9094"</code>
<code>KAFKA_NODE_ID</code>	<code>2</code>	<code>3</code>
El puerto PLAINTEXT interno	<code>29093</code>	<code>29094</code>
El puerto CONTROLLER interno	<code>39093</code>	<code>39094</code>
El puerto EXTERNAL	<code>9093</code>	<code>9094</code>
El volumen	<code>kafka-broker-2-data</code>	<code>kafka-broker-3-data</code>

Y DOS QUE NO SE CAMBIAN

El `CLUSTER_ID` y el `KAFKA_CONTROLLER_QUORUM_VOTERS` van **exactamente iguales en los tres**. Es el error más común de este laboratorio, y produce un clúster que arranca a medias.

Así queda el bloque de listeners del servidor 2, para que compares:

```
KAFKA_LISTENERS: 'PLAINTEXT://kafka-broker-2:29093,CONTROLLER://kafka-broker-2:39093,EXTERNAL://0.0.0.0:9093'
KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://kafka-broker-2:29093,EXTERNAL://localhost:9093'
```

Cambio 4 · los volúmenes

Al final del archivo hay tres líneas, dos comentadas. **Descoméntalas:**

```
volumes:
  kafka-broker-1-data:
  kafka-broker-2-data:
  kafka-broker-3-data:
```

Guarda con **Ctrl+S**.

VAS BIEN SI...

Tu archivo tiene **tres bloques de servicio** y **tres volúmenes**. Compruébalo sin abrir el editor:

```
grep -cE "^ kafka-broker-[123]:" mi-cluster/docker-compose.yml
grep -c "CLUSTER_ID" mi-cluster/docker-compose.yml
```

El primero tiene que decir **3** y el segundo también **3**.

SI TE ATASCAS, HAY UNA SOLUCIÓN DE REFERENCIA

Está en `soluciones/docker-compose-3-brokers.SOLUCION.yml` . **Míralo después de intentarlo**, no antes. Y si lo usas, cámbiale el `CLUSTER_ID` por el tuyo.

Encender los tres

El comando

```
cd mi-cluster
docker compose -p novatech-lab02 up -d
cd ..
```

cd mi-cluster — entra a la carpeta donde está tu plano

docker compose up -d — crea y levanta todo lo que el plano declara. La **-d** es *detached*, o sea que corre en segundo plano y te devuelve la terminal

-p novatech-lab02 — **el nombre del proyecto**. Delimita qué puede borrar un **down -v** más adelante, para que no se lleve por delante los volúmenes de otro laboratorio

cd .. — vuelve a la raíz del laboratorio, desde donde se ejecuta todo lo demás

LO QUE VAS A VER

```
✓ Network mi-cluster_mi-cluster-network Created
✓ Volume novatech-lab02_kafka-broker-1-data Created
✓ Volume novatech-lab02_kafka-broker-2-data Created
✓ Volume novatech-lab02_kafka-broker-3-data Created
✓ Container kafka-broker-1 Started
✓ Container kafka-broker-2 Started
✓ Container kafka-broker-3 Started
```

Comprueba que están vivos

Espera unos 30 segundos, porque los tres tienen que encontrarse y elegir jefe de turno.

```
docker ps --filter name=kafka-broker
```

LO QUE VAS A VER · RECORTADO A LO ANCHO

NAMES	STATUS	PORTS
kafka-broker-2	Up 7 seconds	0.0.0.0:9093->9093/tcp
kafka-broker-3	Up 7 seconds	0.0.0.0:9094->9094/tcp
kafka-broker-1	Up 7 seconds	0.0.0.0:9092->9092/tcp

VAS BIEN SI...

Salen **tres líneas**, con `Up` en `STATUS`, y en `PORTS` los tres puertos distintos **9092, 9093 y 9094**.

Si sale menos de tres, no sigas. El paso 4 no va a decir nada útil.

SI UNO DICE `RESTARTING` O DESAPARECE DE LA LISTA

Se está muriendo en bucle. Mira por qué:

```
docker logs kafka-broker-2 2>&1 | tail -20
```

Las tres causas, por frecuencia:

Dice `InconsistentClusterIdException`. Un servidor tiene un `CLUSTER_ID` distinto de los otros. Revisa que los tres sean idénticos, carácter por carácter.

Dice algo sobre `voters` o `quorum`. El `KAFKA_CONTROLLER_QUORUM_VOTERS` no es igual en los tres, o tiene un puerto mal. Tiene que ser `1@kafka-broker-1:39092,2@kafka-broker-2:39093,3@kafka-broker-3:39094` en los tres.

Dice `Address already in use`. Dos servidores tienen el mismo puerto en el `KAFKA_LISTENERS`. Cada uno lleva los suyos.

SI SALE `PORT IS ALREADY ALLOCATED` AL LEVANTAR

Otro laboratorio del curso está arriba y ocupa el 9092. Bájalo con `docker compose -p novatech-labNN down -v`, poniendo el número que aparezca en `docker ps`.

SI EL `UP` FALLA CON UN ERROR DE YAML

El YAML es sensible a la indentación. **Los bloques de los tres servidores tienen que estar al mismo nivel**, con dos espacios desde el margen. Si copiaste y pegaste, revisa que no se te haya colado un espacio de más al principio de alguna línea.

Leer el quórum

Qué vamos a hacer

Los tres están arriba. Ahora hay que preguntarle al clúster **quién es el jefe de turno y desde cuándo**, que es la consulta que vas a hacer en producción cada vez que algo vaya raro.

El comando

```
bin/check-quorum.sh
```

bin/check-quorum.sh — un script del curso. Por dentro ejecuta **dos** consultas, `describe --status` y `describe --replication`, y les agrega una ficha que explica cada campo

LOS COMANDOS REALES, LOS QUE VAS A ESCRIBIR EN SUNAT

```
kafka-metadata-quorum --bootstrap-server kafka-broker-1:29092 \
  describe --status

kafka-metadata-quorum --bootstrap-server kafka-broker-1:29092 \
  describe --replication
```

kafka-metadata-quorum — la herramienta que responde por el quórum. **Es la que reemplazó a todo lo que antes se le preguntaba a ZooKeeper**

describe --status — el resumen. Quién manda y desde cuándo. **Solo consulta**

describe --replication — el detalle por nodo. Cuánto le falta a cada uno

LA PRIMERA MITAD · EL ESTADO

```
ClusterId:      YlCN8AhrTfWKst5eUgWhrg
LeaderId:       1
LeaderEpoch:   1
HighWatermark:  27
MaxFollowerLag: 0
MaxFollowerLagTimeMs: 0
CurrentVoters:  [{"id": 1, "endpoints": ["CONTROLLER://kafka-broker-1:39092"]},
                  {"id": 2, "endpoints": ["CONTROLLER://kafka-broker-2:39093"]},
                  {"id": 3, "endpoints": ["CONTROLLER://kafka-broker-3:39094"]}
CurrentObservers: []
```

Campo	Qué dice
ClusterId	El identificador que generaste en el paso 1. Compruébalo, tiene que ser el tuyo
LeaderId	Quién es el jefe de turno ahora mismo. A ti te puede salir el 2 o el 3 , porque la elección es entre iguales
LeaderEpoch	El número de turno. Sube en uno cada vez que hay una elección nueva. Es el campo del paso 6
HighWatermark	Hasta dónde llegó el registro de metadatos que los tres comparten. Avanza solo, con el tiempo
CurrentVoters	Los tres que tienen derecho a votar. Es la lista que escribiste en el paso 2
CurrentObservers	Nodos que aportan capacidad pero no votan . Aquí está vacío. En el laboratorio 08 vas a agregar uno

LA SEGUNDA MITAD · LA REPLICACIÓN

NodeId	DirectoryId	LogEndOffset	Lag	LastFetchTimestamp	Status
1	AAAAAAAAAAAAAAAAAAAAA	30	0	1787850336962	Leader
2	AAAAAAAAAAAAAAAAAAAAA	30	0	1787850336885	Follower
3	AAAAAAAAAAAAAAAAAAAAA	30	0	1787850336885	Follower

Columna	Qué dice
Status	Un Leader y dos Follower . Es el reparto sano
LogEndOffset	Hasta dónde tiene copiado cada uno el registro de metadatos
Lag	Cuánto le falta a cada seguidor para alcanzar al líder. En 0 significa al día
LastFetchTimestamp	Cuándo fue la última vez que ese nodo pidió novedades. Un valor antiguo aquí es la señal de un nodo que se quedó atrás

VAS BIEN SI...

En **CurrentVoters** hay **tres** identificadores, y en la tabla de replicación hay **un Leader** y **dos Follower**, los tres con **Lag** en 0.

Anota el LeaderId y el LeaderEpoch. Los vas a comparar en el paso 6.

SI `CURRENTVOTERS` TRAE MENOS DE TRES

Uno de los servidores no se unió al quórum. Casi siempre su `KAFKA_CONTROLLER_QUORUM_VOTERS` es distinto del de los otros. Bájalo todo con `cd mi-cluster && docker compose -p novatech-lab02 down -v && cd ..`, corrige el archivo y vuelve a levantar.

Un tópico con tres copias

Qué vamos a hacer

Con un solo servidor, en el laboratorio 01, no pudiste crear un tópico con tres copias. Kafka te dijo que no. **Ahora sí se puede**, y conviene ver qué cambia.

CONCEPTO · RÉPLICA E ISR

Una **réplica** es cada una de las copias de una partición, viviendo en un servidor distinto. Son las libretas de respaldo.

El **ISR** (*In-Sync Replicas*) es **cuáles de esas copias están al día en este momento**. Son los que marcaron tarjeta. **Esta es la columna que vas a mirar en el paso 6**, cuando falte uno.

El comando

```
docker exec kafka-broker-1 kafka-topics \  
  --bootstrap-server kafka-broker-1:29092 \  
  --create --topic novatech.test \  
  --partitions 6 --replication-factor 3
```

docker exec kafka-broker-1 — ejecuta el comando **dentro** del primer servidor. **En un servidor de verdad esta parte no existe**

--bootstrap-server kafka-broker-1:29092 — el puerto **interno**, porque estamos hablando desde dentro de la red de Docker

--partitions 6 — en cuántos pedazos se corta. **Seis, para que haya de sobra que repartir entre tres servidores**

--replication-factor 3 — tres copias de cada pedazo, en tres servidores distintos. **Esto es lo que en el laboratorio 01 fallaba**

Ahora descríbelo

```
docker exec kafka-broker-1 kafka-topics \  
  --bootstrap-server kafka-broker-1:29092 \  
  --describe --topic novatech.test
```

LAS PRIMERAS LÍNEAS · RECORTADO A LO ANCHO

```
Topic: novatech.test PartitionCount: 6 ReplicationFactor: 3
Configs: min.insync.replicas=2
Partition: 0 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1
Partition: 1 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2
Partition: 2 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
```

Fíjate en dos cosas. Cada partición tiene un líder distinto, así que el trabajo está repartido entre los tres. Y en las tres, `Isr` trae **los mismos tres números** que `Replicas`, o sea que las tres copias están al día.

VAS BIEN SI...

`ReplicationFactor: 3`, seis líneas de partición, y en todas `Isr` **tiene tres números**.

Copia esas seis líneas a un papel. En el paso 6 vas a comparar contra ellas.

ESE `MIN.INSYNC.REPLICAS=2` QUE APARECE SIN QUE LO PIDIERAS

Viene del plano, del `KAFKA_MIN_INSYNC_REPLICAS: 2`. Significa que **para aceptar una escritura hacen falta al menos dos copias al día**. Con tres servidores puedes perder uno y seguir escribiendo. **Si perdieras dos, el tópico dejaría de aceptar escrituras**, y eso es a propósito. Prefiere no aceptar el dato antes que aceptarlo sin respaldo.

Matar al jefe de turno

Qué vamos a hacer

Aquí está el laboratorio. Vas a apagar **el servidor que manda**, no uno cualquiera, y a mirar qué hacen los otros dos.

Primero, mira quién es

```
docker exec kafka-broker-1 kafka-metadata-quorum \
  --bootstrap-server kafka-broker-1:29092 describe --status | head -3
```

LO QUE SALIÓ EN LA CORRIDA MEDIDA

```
ClusterId:          YlCN8AhrTfWKst5eUgWhrg
LeaderId:           2
LeaderEpoch:       3
```

Anota los dos números. En este ejemplo el jefe de turno es el **2** y el turno va por el **3**. A ti te van a salir otros.

Ahora apágalo

```
docker stop kafka-broker-2
```

docker stop — apaga el contenedor **sin borrarlo**. Su disco queda intacto, y por eso lo vas a poder encender de nuevo en el paso 7

kafka-broker-2 — **pon el número que te salió a ti**, no el 2 del ejemplo

ESPERA 20 SEGUNDOS ANTES DE PREGUNTAR

Los otros dos tienen que **notar que falta**, convocar la elección y ponerse de acuerdo. Si preguntas de inmediato vas a ver un clúster a medio camino.

Pregúntale a uno de los que quedan

OJO CON EL PUERTO INTERNO · CADA SERVIDOR TIENE EL SUYO

Servidor	Puerto interno
kafka-broker-1	29092
kafka-broker-2	29093
kafka-broker-3	29094

```
docker exec kafka-broker-1 kafka-metadata-quorum \
  --bootstrap-server kafka-broker-1:29092 describe --status
```

LO QUE SALIÓ · EL CLÚSTER SIN EL 2

```
ClusterId:      YlCN8AhrTfWKst5eUgWhrg
LeaderId:       3
LeaderEpoch:   4
HighWatermark:  558
MaxFollowerLag: 559
MaxFollowerLagTimeMs: -1
CurrentVoters:  [{"id": 1, ...}, {"id": 2, ...}, {"id": 3, ...}]
CurrentObservers: []
```

Cómo se lee, y son tres cosas

Uno. Cambió el jefe de turno. Era el 2 y ahora es el 3. **Nadie ejecutó un comando de elección.** Los dos que quedaban lo hicieron solos.

Dos. Subió el número de turno. El `LeaderEpoch` pasó de 3 a 4. Ese número es la manera que tiene Kafka de saber qué órdenes son del turno actual y cuáles son de un turno viejo. **Si un servidor que estuvo desconectado vuelve dando órdenes del turno 3, se las ignoran.**

Tres. `CurrentVoters` sigue trayendo los tres. Y eso confunde a mucha gente. **La lista de votantes es la definición del quórum, no la lista de quiénes están vivos.** Para saber quién está vivo se mira el `MaxFollowerLag`, que aquí saltó a 559, y el `MaxFollowerLagTimeMs` en `-1`. Ese `-1` significa «hay uno del que no sé nada».

Y comprueba que el clúster sigue atendiendo

```
docker exec kafka-broker-1 kafka-topics \  
  --bootstrap-server kafka-broker-1:29092 \  
  --describe --topic novatech.test | head -4
```

LO QUE SALIÓ

```
Topic: novatech.test  PartitionCount: 6  ReplicationFactor: 3  
  Partition: 0  Leader: 3  Replicas: 2,3,1  Isr: 1,3  
  Partition: 1  Leader: 3  Replicas: 3,1,2  Isr: 1,3  
  Partition: 2  Leader: 1  Replicas: 1,2,3  Isr: 1,3
```

ESTA SALIDA DICE TRES COSAS A LA VEZ

El tópico responde. El comando devolvió datos, así que el clúster está atendiendo con dos de tres servidores.

El `Isr` se encogió, de tres números a dos. El 2 salió de la lista de los que están al día, porque está apagado.

Y el liderazgo se movió. Mira la partición 0. Sus réplicas son `2,3,1` y el líder era el 2. Ahora el líder es el 3. **El trabajo del que se cayó lo tomó otro, partición por partición.**

LO LOGRASTE SI...

El `LeaderId` cambió y el `LeaderEpoch` subió en uno. Y el `describe` del tópico sigue respondiendo, con el `Isr` encogido a dos.

Eso es la primera mitad de la afirmación. **El clúster aguantó perder uno.**

SI EL COMANDO SE QUEDA COLGADO REPITIENDO `CONNECTION TO NODE -1 COULD NOT BE ESTABLISHED`

Le estás preguntando **al servidor que apagaste**, o al puerto interno equivocado.

Corta con `Ctrl+C` y repite contra uno que siga vivo, **con su puerto**, según la tabla de arriba. Preguntarle al 2 por el 29092 falla aunque el 2 esté vivo, porque el 29092 es del 1.

SI EL LEADEREPOCH NO CAMBIÓ

Apagaste un seguidor, no al líder. **No es un error**, y de hecho demuestra algo útil: el clúster ni se inmuta cuando cae uno que no manda. Enciéndelo de nuevo con `docker start`, vuelve a mirar quién es el `LeaderId`, y apaga a ese.

Devolverlo, y ver que no recupera el puesto

Qué vamos a hacer

El servidor vuelve. La pregunta que casi todo el mundo contesta mal es **si recupera el liderazgo que tenía**.

Piénsalo antes de ejecutar. Era el jefe de turno, tiene todos los datos, su disco está intacto. ¿Vuelve a mandar?

El comando

```
docker start kafka-broker-2
```

Espera unos 40 segundos y pregunta otra vez:

```
bin/check-quorum.sh
```

LO QUE SALIÓ

```
ClusterId:      YlCN8AhrTfWKst5eUgWhrg
LeaderId:       3
LeaderEpoch:   4
HighWatermark:  680
MaxFollowerLag: 0
MaxFollowerLagTimeMs: 492
```

NodeId	DirectoryId	LogEndOffset	Lag	Status
3	AAAAAAAAAAAAAAAAAAAAA	683	0	Leader
1	AAAAAAAAAAAAAAAAAAAAA	683	0	Follower
2	AAAAAAAAAAAAAAAAAAAAA	683	0	Follower

LA RESPUESTA ES NO, Y MIRA LOS TRES NÚMEROS

El **LeaderId** sigue siendo **3**. El 2 volvió y no reclamó nada.

El **LeaderEpoch** sigue en **4**. No hubo elección nueva, porque no hacía falta. Ya había jefe de turno.

El 2 volvió como **Follower**, con **Lag** en **0**. Se puso al día en segundos y se sentó a copiar.

Por qué esto es deseable

QUÉ SIGNIFICA ESTO PARA SUNAT

Un liderazgo que vuelve solo al servidor original suena ordenado y es **una mala idea**. Significaría una segunda elección, y una segunda pausa, cada vez que un servidor se reinicia.

Un servidor que se reinicia dos veces en una hora provocaría **cuatro** elecciones en vez de dos.

Kafka prefiere la estabilidad al orden.

Y de ahí sale la pregunta que hay que hacer en producción. Cuando alguien diga «reiniciamos el broker 2 y todo volvió a la normalidad», la pregunta correcta es «**¿volvió a ser líder, o volvió como seguidor?**». Las dos son normales. Solo una de las dos costó una pausa.

Y comprueba que el tópico se recuperó

```
docker exec kafka-broker-1 kafka-topics \  
  --bootstrap-server kafka-broker-1:29092 \  
  --describe --topic novatech.test | head -4
```

LO QUE SALIÓ

```
Topic: novatech.test  PartitionCount: 6  ReplicationFactor: 3  
  Partition: 0  Leader: 3  Replicas: 2,3,1  Isr: 1,2,3  
  Partition: 1  Leader: 3  Replicas: 3,1,2  Isr: 1,2,3  
  Partition: 2  Leader: 1  Replicas: 1,2,3  Isr: 1,2,3
```

LO LOGRASTE SI...

El **Isr** volvió a tener tres números, o sea que el 2 se puso al día. Y sin embargo, **la partición o sigue liderada por el 3** aunque su primera réplica sea el 2.

Eso cierra el laboratorio. **Los datos se recuperan solos. El liderazgo no vuelve solo.**

Lo que aprendiste

1 Tres servidores aguantan perder uno. Dos, no.

La elección necesita mayoría, y de tres la mayoría es dos. Por eso los quóruns se dimensionan en números impares, con tres o cinco. Un quórum de dos no tolera ninguna caída, porque uno solo nunca es mayoría de dos.

2 `CurrentVoters` no dice quién está vivo.

Dice quién tiene derecho a votar, que es la lista que tú escribiste. Para saber quién está vivo se mira el `Status` de cada nodo, su `Lag` y su `LastFetchTimestamp`. Confundir las dos cosas es el error de lectura más común de esta salida.

3 El `LeaderEpoch` es el contador de elecciones.

Sube en uno cada vez que hay una. Es lo que le permite al clúster ignorar las órdenes de un servidor que estuvo desconectado y vuelve creyendo que todavía manda.

4 El que vuelve entra como seguidor.

Los datos se recuperan solos y el liderazgo no. Es una decisión de diseño a favor de la estabilidad, porque devolver el liderazgo costaría una segunda elección y una segunda pausa cada vez que alguien reinicia un servidor.

Para profundizar

Lo que sigue no está en el recorrido de clase. **El clúster que tienes arriba sirve para todo esto.**

A · Leerle el disco a cada servidor

```
bin/verify-storage.sh kafka-broker-1
bin/verify-storage.sh kafka-broker-2
bin/verify-storage.sh kafka-broker-3
```

LO QUE SALE PARA EL PRIMERO

```
Found log directory:
  /var/lib/kafka/data
Found metadata: {cluster.id=YlCN8AhrTfWKst5eUgWhrg,
  directory.id=56uijmDgA69Xqc7-LoodWg, node.id=1, version=1}
```

Lo que hay que mirar. Corre los tres y compara. El `cluster.id` es el mismo en los tres, porque es la identidad compartida. El `node.id` y el `directory.id` son **distintos en cada uno**. Es la comprobación que harías en un servidor real cuando sospechas que alguien copió un directorio de datos de una máquina a otra.

B · Apagar dos, y ver qué deja de funcionar y qué no

Es la contraparte del paso 6, y demuestra la regla 1. **Lo interesante es que no se cae todo de golpe.**

```
docker stop kafka-broker-2 kafka-broker-3
```

Espera 20 segundos y prueba **tres cosas distintas**, en este orden.

Uno. Leer lo que ya existe.

```
docker exec kafka-broker-1 kafka-topics \
  --bootstrap-server kafka-broker-1:29092 --list
```

LO QUE SALE

```
novatech.test
```

Funciona. Y sorprende a todo el mundo. El servidor que queda tiene su propia copia de los metadatos, así que puede contestar de memoria qué tópicos existen.

Dos. Crear algo nuevo.

```
docker exec kafka-broker-1 kafka-topics \
  --bootstrap-server kafka-broker-1:29092 \
  --create --topic novatech.imposible \
  --partitions 1 --replication-factor 1
```

LO QUE SALE, TRAS UN RATO REINTENTANDO

```
WARN [AdminClient clientId=adminclient-1] Error connecting to node
kafka-broker-3:29094 (id: 3 rack: null isFenced: false)
java.net.UnknownHostException: kafka-broker-3: Name or service not known
```

Falla. Crear un tópicos es **escribir** en el registro de metadatos, y para eso hace falta que la mayoría esté de acuerdo. Con uno de tres no hay mayoría.

Tres. Preguntar por el quórum.

```
docker exec kafka-broker-1 kafka-metadata-quorum \
  --bootstrap-server kafka-broker-1:29092 describe --status
```

Se cuelga. Esa consulta se la hace al jefe de turno, y no hay. Córdala con `Ctrl+C`.

LA REGLA QUE SALE DE ESTAS TRES PRUEBAS

Sin quórum, el clúster puede seguir contestando lo que ya sabe, y no puede aprender nada nuevo. Por eso un clúster sin mayoría puede parecer sano desde un tablero que solo hace consultas de lectura. **La prueba honesta es intentar escribir.**

Y ESE COMPORTAMIENTO ES CORRECTO

Un servidor solo **no puede saber** si los otros dos están muertos o si es él quien perdió la red. Si siguiera atendiendo por su cuenta y los otros dos siguieran vivos al otro lado, tendrías **dos clústeres escribiendo el mismo dato. Kafka prefiere no atender antes que arriesgarse a eso.**

Para volver:

```
docker start kafka-broker-2 kafka-broker-3
```

C · Mirar quién manda en cada partición

```
docker exec kafka-broker-1 kafka-topics \  
  --bootstrap-server kafka-broker-1:29092 \  
  --describe --topic novatech.test
```

Y para contarlo sin hacerlo a ojo:

```
docker exec kafka-broker-1 kafka-topics \  
  --bootstrap-server kafka-broker-1:29092 \  
  --describe --topic novatech.test | grep -oE "Leader: [0-9]+" | sort | uniq -c
```

LO QUE SALIÓ EN LA CORRIDA MEDIDA

```
2 Leader: 1  
2 Leader: 2  
2 Leader: 3
```

Lo que hay que mirar. Dos particiones por servidor, repartidas parejo. Kafka distribuye el liderazgo al crear el tópico, y ese reparto es lo que hace que el trabajo no caiga todo en la misma máquina. **Si acabas de hacer el paso 6, el reparto va a estar desbalanceado**, porque el que se cayó perdió sus liderazgos y no los recuperó.

D · Los archivos de configuración de un contenedor vivo

Es un adelanto del laboratorio 03, y se ve en dos comandos.

```
docker exec kafka-broker-1 sh -c "ls /etc/kafka/*.properties | wc -l"  
docker exec kafka-broker-1 sh -c "ps -o args= -C java | tr ' ' '\n' | grep properties"
```

LO QUE SALE

```
13  
/etc/kafka/kafka.properties
```

Lo que hay que mirar. Hay **trece** archivos `.properties`, y uno de ellos se llama `server.properties`. **Pero el proceso no arrancó con ese.** Arrancó con `kafka.properties`, que **no lo escribió nadie**. Lo genera la imagen al arrancar, traduciendo las variables `KAFKA_*` que tú escribiste en el plano.

La misma imagen, apagada, tiene **doce**. El decimotercero solo existe cuando el contenedor corre. **Ese es el tema entero del laboratorio 03.**

E · El reporte del laboratorio

`plantillas/reporte-entregable.md` , con las respuestas de referencia en `soluciones/reporte-resuelto.md` . **Míralas después de intentarlo.**

Antes de cerrar

DEJA EL TERRENO LIMPIO

Si vas a hacer otro laboratorio, baja este primero o los puertos van a estar ocupados:

```
cd mi-cluster
docker compose -p novatech-lab02 down -v
cd ..
```

Esa `-v` borra también los volúmenes, o sea los tres discos. Si quieres conservarlos para seguir mañana donde lo dejaste, usa `bin/stop-mi-cluster.sh` , que apaga sin borrar.

Y comprueba con `docker ps --filter name=kafka-broker` que no quede nada.

LO QUE TE LLEVAS

Ya sabes leer las dos salidas con las que se diagnostica un clúster KRaft. Y sabes las dos preguntas que hay que hacer cuando alguien reporta que un servidor se cayó y volvió. **¿Cambió el LeaderEpoch ? Y ¿el Isr recuperó sus tres números?**